

Response to the referees

cnefetools: Access and Analysis of Brazilian CNEFE Address Data in R (submission 2026-37)

Jorge Ubirajara Pedreira Junior and Bruno Henrique Miotto Stabile

28 September 2026

Dear Dr Tanaka,

Thank you for handling our submission, and our thanks to both referees for careful and constructive reports. Between them they covered the methods, the code and the text, and several of their comments led to improvements we would not otherwise have made. We checked each comment against the package and the manuscript, reran the measurements where a comment depended on them, and revised the package, its documentation and the article accordingly.

This document reproduces every comment in full, followed by our response. We've numbered the comments so the responses can refer to each other (R1.1 to R1.13 for Referee 1, and R2.1 to R2.12 and R2.C1 to R2.C10 for Referee 2), and section numbers refer to the revised manuscript. Where we decided not to follow a suggestion, the response explains why. The changes to the package are in the development version on GitHub (<https://github.com/pedreirajr/cnefetools>) and will be released on CRAN as version 0.3.0.

Referee 1

General

Evaluating the manuscript and accompanying software against [The R Journal guidelines](#), `cnefetools` demonstrates strong adherence. The paper balances geospatial context with a clear focus on the software's design, dual DuckDB/R backend, and performance benchmarks. The package addresses the data engineering challenge of processing 110 million Brazilian address records, making it a critical resource for researchers focused on Brazilian geospatial analysis. Beyond its immediate utility, `cnefetools` serves as a technical blueprint for the R developer community, demonstrating a performant architecture for the emerging class of DuckDB-powered data access packages. I have independently verified the examples in `read_cnefe()` and the primary vignettes; the package is fully functional and the API is intuitive. Furthermore, the codebase follows modern R development standards, including semantic messaging via `cli` and comprehensive `roxygen2` documentation.

Response: Thank you. We're glad the referee ran the examples and the articles independently, and we've tried to keep the package to that standard throughout this revision.

Methodological considerations: land use mix indices

R1.1: Binary classification and loss of granularity

The core analytical module of the package, `compute_lumi()`, is built around a binary classification: Residential (Type 1) versus Non-Residential (Types 2–8). While the manuscript (Section 4) and the referenced preprint (Pedreira Junior et al. 2025) are transparent about this choice, the implementation results in a significant loss of the “fine-grained” information that the authors initially promote as a key advantage of the CNEFE dataset. Collapsing eight distinct categories—ranging from schools and hospitals to religious temples and factories—into a single “Non-Residential” bin means that diversity indices like **Entropy** and **HHI** do not actually measure functional land-use diversity, but rather simple “residential dominance.” As a result, a neighborhood with ten different types of services would receive a lower “mix” score than one with a single service and a few houses, which is counter-intuitive for urban planning research and deviates from the multi-category standards discussed in the cited literature (e.g., Song et al. 2013).

Response: We keep the binary formulation and now state clearly what it measures.

We agree with the referee that with two categories the Entropy Index and the HHI measure how evenly an area is split between housing and everything else, not how diverse its non-residential uses are. That is the construct the indices are meant to capture, and it’s the one on which they were defined and validated against job accessibility in Pedreira Junior et al. (2026), now published in *Land Use Policy*. Redefining the indices inside a software paper would change the object the paper describes, so we haven’t done that.

We have, however, made this much more explicit. Section 4.3 now says that the indices rest on a binary split, that they measure the balance between housing and all other uses rather than the diversity of those uses, and that an area whose non-residential addresses are all of one type scores the same as one where they are spread across every type, provided both have the same residential share. It also acknowledges that this differs from the multi-category definitions common in urban research, citing Song et al. (2013), and points users who need that diversity to the per-type counts that `cnefe_counts()` returns. Section 6 discusses where the binary split falls short, with the examples Referee 2 gives in R2.1, and multi-category formulations stay on the list of future work.

R1.2: Reliance on Unreviewed Methodology

Reliance on Unreviewed Methodology: The package serves as a “pixel-perfect” implementation of the authors’ own directional indices (aHHI and BGBI), which are currently documented in an unreviewed engrXiv preprint (Pedreira Junior et al. 2025). While this demonstrates high internal consistency between the paper and the code, it also positions the package as a highly specialized tool for one specific methodological framework rather than a general-purpose geospatial utility. Given that these indices (specifically the BGBI) rely on a nonlinear transformation centered on a global proportion, their scientific validity rests entirely on the unreviewed preprint’s claims. For a research-focused package, this narrow methodological scope should be more explicitly signaled to the user.

Response: The BGBI is now published in a peer-reviewed journal; the adapted HHI remains in the preprint, and the manuscript distinguishes the two.

The BGBI has since been peer-reviewed and published (Pedreira Junior et al., 2026, *Land Use Policy* 169, 108135), where it is computed alongside four established indices and validated against job accessibility measures. The manuscript now cites that article for it.

The adapted HHI is still documented only in the engrXiv preprint (Pedreira Junior et al., 2025), which the manuscript keeps and now labels as a preprint. It is a monotone rescaling of the standard HHI, which runs from 0.5 to 1 with two categories, into the range 0 to 1, with a sign that shows which use dominates. It inherits the properties of an established index and adds only a direction, so it doesn't carry a validity claim of its own.

On scope, three of the six indices (entropy, HHI and the Balance Index) are standard measures from the literature, and the Index of Concentration at Extremes comes from Massey (2001), so only the BGBI and the adapted HHI are ours. The methodological basis of the indices is now explicit in Section 4.3 and in `?compute_lumi`, as described under R1.3.

R1.3: Ambiguity in Documentation

Ambiguity in Documentation: The user-facing documentation (`?compute_lumi`) fails to mention the binary simplification. A user reading the package's description of the 8 CNEFE categories would naturally assume the "Entropy Index" utilizes all available data. This creates a significant risk of misinterpretation: a researcher might publish results claiming "high land-use diversity" in a zone that is actually functionally homogeneous but balanced between residential and one other use. I recommend that the authors:

1. Update the function documentation to explicitly warn about the binary split.
2. Provide a theoretical justification for why the binary split was chosen over a multi-category approach, especially given that the raw CNEFE data already provides the necessary granularity. This is covered in the preprint, but a brief summary in the package documentation and the package paper would help users understand the rationale without needing to read the full paper.
3. Acknowledge that while this logic is consistent with their preprint, it may not align with the prevalent multi-category definitions of Land Use Mix in broader urban science.

Response: Accepted on all three points.

`?compute_lumi` now has a `@details` section, which it previously lacked. On point 1, it states the rule: an address is counted as residential when `COD_ESPECIE == 1` and as non-residential otherwise. On point 2, the justification now cites the peer-reviewed article rather than the preprint, since the indices are defined and empirically validated on that two-category basis in Pedreira Junior et al. (2026), *Land Use Policy* 169, 108135. The same `@details` block also documents the type 7 exclusion raised in R2.2 and the citywide baseline P raised in R2.3, so the help page and the article now describe the same behaviour.

While checking the exclusion in order to document it, we also found a defect in one code path, reported under R2.2 below.

The manuscript carries the same three points in Section 4.3: the rule, the justification with its citation, and the acknowledgement that the formulation differs from the multi-category definitions common in urban research (Song et al., 2013). It also tells users who need the diversity of non-residential uses that `cnefe_counts()` returns counts for all eight address types.

Methodological considerations: H3 grids

R1.4: H3 is not equal-area, and equal-area alternatives deserve a mention

While providing H3 hexagonal aggregation is a practical and powerful feature, the H3 system (originally developed by Uber for logistical purposes) has notable limitations for rigorous spatial analysis. Because it uses a polyhedral gnomonic projection, it is not an equal-area grid. Cell areas can vary significantly, with distortions reported up to 50% across the globe. This is particularly relevant for a country with the vast latitudinal extent of Brazil. As Kmoch et al. (2025) emphasises, these distortions can introduce analytical biases in density modelling or aggregations where area uniformity is assumed.

The authors demonstrate an excellent level of methodological rigour when discussing interpolation methods (Introduction and Section 4). They clearly articulate why dasymetric interpolation using CNEFE dwelling points as ancillary data is superior to simpler areal-weighted approaches. The internal implementation of these spatial operations is also robust, automatically reprojecting user-supplied polygons to a consistent CRS (EPSG:4326) to prevent silent join failures. I believe the discussion would benefit from a similar level of rigour regarding the choice of H3 as the default grid system.

By including H3 as a primary built-in option, `cnefetools` understandably provides a practical solution for many users. However, it is also helpful to keep in mind that spreading awareness of H3's spatial drawbacks can prevent unintentional analytical biases. While it is entirely up to the authors' discretion whether to expand on this in the manuscript or documentation, I believe that briefly mentioning equal-area alternatives could be a valuable service to the community. Equal-area alternatives such as **ISEA**, the **A5** grid (available via the [a5R](#) package), or the newer **IGEO7** system (Kmoch et al. 2025) help users make more informed decisions when high-precision area consistency is required.

Response: Accepted. The manuscript now notes that H3 is not equal-area and points to alternatives.

We thank the referee for this suggestion. H3 uses a gnomonic projection onto an icosahedron, so cell areas are not constant, and across a country the size of Brazil this should be stated explicitly.

Section 4.2, right where the manuscript quotes the average cell area, now carries a short paragraph noting that H3 is not equal-area, that this matters whenever an analysis assumes cells of equal size, as in density estimates, and pointing readers to equal-area alternatives when that is a requirement, namely the A5 grid through the `a5R` package (Graham, 2026), which is on CRAN and now cited, and IGEO7, which is built on the ISEA projection (Kmoch et al., 2025).

To give a figure relevant to Brazil, we measured the variation across the country. Across Brazil, resolution 9 cells range from about 80,000 m² to 127,000 m², so the largest is about 1.6 times the smallest, and 1.47 times between the 1st and 99th percentiles. The manuscript reports that range. It comes from the areas, computed with `h3jsr::cell_area()`, of the 19,998 distinct cells hit by a random sample of 20,000 points inside the national boundary.

We keep H3 as the built-in option, and the manuscript now says why. The DuckDB `h3` extension assigns cells inside the query, which keeps the aggregation in-process, and H3 cell identifiers can be joined with other data indexed on the same grid.

Minor comments

R1.5: Inconsistent software citation

The authors are inconsistent in citing software used or discussed in the manuscript. While they cite `sf`, `areal`, `geobr`, `populR`, `duckdb`, and `h3jsr`, implementation-relevant packages such as `arrow`, `duckspatial`, and `piggyback` are mentioned without formal citations. The model-based change-of-support package `smile` is also discussed in the related-work section without a citation. These omissions should be corrected to ensure proper attribution of open-source software.

Response: Accepted. This was an oversight on our part.

All four packages are on CRAN and therefore have canonical citations. `arrow`, `duckspatial`, `piggyback` and `smile` now have bibliography entries taken from each package's own citation, and each is cited at its first mention in the text. We also added citations for `censobr`, which the new provenance statement relies on (R1.9), and for `a5R` (R1.4).

R2.11 raises the `smile` citation independently and is answered by the same change.

Code: usability

R1.6: Workflow & API: `polygon_type` is redundant

Workflow & API: The package has a highly intuitive and user-friendly workflow. The dual-backend architecture is abstracted cleanly behind a single `backend = "duckdb"` or `"r"` argument. However, the `polygon_type` argument feels somewhat redundant. If a user passes an `sf` object to the `polygon` argument, it naturally implies they want to use custom polygons rather than an H3 grid. While the package currently infers this and issues a warning, the authors could consider making this inference automatic and silent. Alternatively, they could deprecate `polygon_type` entirely in favour of simply checking whether `polygon` is `NULL` or an `sf` object.

Response: Accepted. `polygon_type` is now deprecated and the mode is inferred from `polygon`.

We agree that `polygon_type` is redundant, since passing an `sf` object to `polygon` already states the intent. Of the two options the referee offers, we chose to deprecate the argument.

The mode is now inferred from whether `polygon` is `NULL`. Code that still passes `polygon_type` keeps working and emits a `lifecycle` deprecation warning, so nothing breaks for existing users. The three alert lines the function used to print when it inferred the mode are gone, since there is no longer an inconsistency to announce.

Two existing behaviours are kept. `polygon_type = "user"` with no `polygon` remains an error, because it states an intent the call cannot satisfy and it errored before. `polygon_type = "hex"` alongside a supplied `polygon` resolves to user polygons, which is what the previous inference did.

R1.7: Cache Location Override

Cache Location Override: The package downloads potentially massive ZIP files (up to 901 MB uncompressed) and hardcodes the cache location to `tools::R_user_dir("cnfetools", "cache")`. While caching is a very useful feature and using this directory aligns with CRAN policies for persistent data, forcing users to store large files on their primary partition can be problematic for those with limited space. The authors might consider allowing users to override this cache path. While packages dealing with large geospatial datasets like `osmextract` rely on environment variables to set the download directory, providing an explicit user-facing function argument (e.g., `cache_dir`) might be more intuitive for novice R users. This ensures they can easily point the download to a larger secondary drive or shared volume.

Response: Accepted. The cache location can now be set by argument or by environment variable.

The cache location resolves from the most specific source to the least:

1. a `cache_dir` argument on the calling function,
2. the `CNEFETOOLS_CACHE_DIR` environment variable,
3. `tools::R_user_dir("cnefetools", which = "cache")`, the previous default.

We implemented both because they serve different users: the referee cites `osmextract`, which uses an environment variable, and also notes that an explicit argument is easier for less experienced users to find. The argument outranks the variable so a single call can be redirected without changing the session. `cache_dir` reaches `read_cnefe()`, `cnefe_counts()`, `compute_lumi()`, `tracts_to_h3()`, `tracts_to_polygon()`, and also `clear_cache_muni()` and `clear_cache_tracts()`.

The CRAN-policy default is unchanged, so users who set neither see the previous behaviour. Section 3 of the manuscript describes the order and why both mechanisms exist, and a new article on the package website, “Cache and exported copies”, shows how to use them.

User feedback and documentation (no action requested)

User Feedback: The extensive use of the `cli` package for informative progress steps, success messages, and structured aborts provides an excellent developer experience.

Documentation: The functions are thoroughly documented with `roxygen2`, clearly explaining parameters, return types, and caching behaviour.

Response: Thank you. The functions and arguments added in this revision follow the same conventions and are documented in the same way.

Code: technical implementation

Organisation and style (no action requested)

Organisation & Style: The code is meticulously organised and adheres strongly to modern Tidyverse conventions. Internal helpers are properly prefixed with `.` and annotated with `@noRd` to keep the exported namespace clean. The use of `rlang::check_installed()` for managing optional dependencies (e.g., `sf`, `duckdb`, `piggyback`) is excellent.

Response: Thank you.

R1.8: Resilience to URL Changes and Data Versioning

Resilience to URL Changes and Data Versioning: The package relies entirely on a pre-built internal index (`cnefe_index_2022`) to resolve IBGE FTP download URLs. If IBGE changes their directory structure or filenames, existing versions of the package will fail to download data. Currently, the package handles failures gracefully via `httr2`. It attempts hardcoded retries (300s, 600s, 1800s) before aborting, but lacks a dynamic fallback mechanism. Furthermore, while the manuscript states that `cnefetools` “targets the 2022 edition and future censuses,”

the current architecture is strictly hardcoded for 2022. The internal `.get_cnefe_index()` and `.validate_year()` functions explicitly abort for any year other than 2022. If the 2030 census introduces a different schema or FTP hierarchy, the package will require a significant update. The authors might consider adding a dynamic fallback that scans the FTP server at runtime to reconstruct URLs or at least acknowledging that support for “future censuses” is currently a roadmap item rather than a present capability. Additionally, adding a pre-flight availability check could vastly improve the user experience. If the base IBGE server is unreachable, inform the user they may have a connectivity issue. If the server is reachable but the specific file is not (e.g., a 404 error), inform the user that the upstream directory structure may have changed and encourage them to report the issue on the package’s issue tracker. I have included a small supplementary script (`test_url_fallback.R`) that illustrates the current behaviour when an internal IBGE URL is broken. It should not be treated as a production test in its current form because it mutates internal package data and can trigger the long retry timeouts, but it may be useful as a starting point for a mocked `testthat` regression test with shorter retry intervals.

Response: This comment contains three suggestions, which we answer separately below, as R1.8a, R1.8b and R1.8c.

R1.8a: Dynamic FTP fallback

Response: We did not add a runtime scan of the FTP server; we address the underlying risk in two other ways.

A dynamic fallback would have to rebuild URLs from a directory layout that has just changed in an unknown way, so it could only guess, and a wrong guess is harder for a user to diagnose than a clear failure. We have instead addressed the concern through the changes described under R1.8c and R1.11. The pre-flight check now tells the user when a file is missing because the layout changed upstream and asks them to report it, so the index can be rebuilt with `data-raw/cnefe_index_2022.R` and shipped in a patch release. Until then, `read_cnefe(file = ...)` reads a ZIP the user downloads by hand, as IBGE publishes it, without going through the index. Section 3 of the manuscript now describes both.

R1.8b: “Future censuses” claim versus hardcoded 2022

Response: Accepted. The grid construction no longer hardcodes the year.

We agree that the claim did not match the hardcoded year. Instead of changing the sentence, we removed the hardcoding. `.read_muni_boundary_2024()`, which called `geobr` with `year = 2024` fixed, is now `.read_muni_boundary(code_muni, year)`, and `build_h3_grid()` gained a `year` argument that `cnefe_counts()`, `compute_lumi()` and `tracts_to_h3()` thread down from their own validated `year`.

The change also fixes a related mismatch. The grid was previously built from a 2024 territorial base while the data aggregated onto it, and in `tracts_to_h3()` the census tracts interpolated onto it, belong to 2022. IBGE geocoded the CNEFE records against the base of their own year, so the grid now shares the reference frame of the data.

In principle this can change results. Measured at resolution 9, the 2022 and 2024 grids are identical for both municipalities used in the package articles, Lauro de Freitas-BA at 550 cells and Fortaleza-CE at 2651 cells, so no published figure moves.

R1.8c: Pre-flight availability check

Response: Accepted. A pre-flight check now distinguishes the failure cases and runs before any retry.

We first checked how the live IBGE server responds, and it answers `HEAD` correctly: 200 with a content-length for a file that exists, 404 for one that does not, and a transport-level failure when the host cannot be resolved. The three cases the referee describes can therefore be told apart at low cost.

- **Server unreachable.** The message names the host and points at connectivity, proxies and firewalls.
- **404.** The internal index is built from the published directory layout, so a 404 on a URL the package generated means that layout changed upstream. The message states that this is a problem with `cnefetools` rather than with the user's setup, and asks them to report it with the URL. Each case also carries a distinct condition class, so downstream code can branch on it.

The check runs before the retry ladder. Previously a URL that could never resolve consumed timeouts of 300, then 600, then 1800 seconds before failing. It now fails in seconds.

On the supplementary script, as the referee cautions, we did not adapt `test_url_fallback.R` directly, since it mutates internal package data and can trigger those long timeouts. The regression test mocks `httr2::req_perform()` instead, so it touches no network and enters no retry. One of its assertions counts requests, confirming that a failed pre-flight is followed by zero download attempts.

The script was very helpful for understanding the failure modes, and we thank the referee for it.

R1.9: External Data Provenance and Reproducibility

External Data Provenance and Reproducibility: In addition to the municipal CNEFE ZIP files downloaded from IBGE, the package relies on derived internal data objects and GitHub-release assets, including the pre-built `cnefe_index_2022` and the UF-level census-tract Parquet files used by `tracts_to_h3()` and `tracts_to_polygon()`. The manuscript describes these assets functionally, but it does not yet give readers enough information to reproduce or audit their generation. I recommend adding a short data provenance statement that identifies the source files, generation scripts, package/data versions, release tags, and checksums or hashes for externally downloaded and derived assets. This is especially important because the package is positioned as infrastructure for reproducible spatial analysis. The authors may find the `geobr` project useful as a model: it uses a reproducible `targets` workflow and supporting infrastructure to generate and maintain its data assets. Adopting a similar approach, or at least documenting the existing asset-generation pipeline more explicitly, would make `cnefetools` more robust to upstream data changes and easier for future contributors to maintain.

Response: Accepted. The census tract assets now have a verified generation script, checksums, and a provenance statement in the manuscript.

The package carried generation scripts in `data-raw/` for `cnefe_index_2022` and for `tracts_variables_ref`, both public in the repository. The UF-level census tract Parquet assets served from GitHub Releases had none. They were built once and published, but the code that built them was never committed, and this was not visible from the manuscript.

Generation script.

`data-raw/sc_assets_build.R` builds every asset from source. Because it was reconstructed from the published files, we checked its output against them tract by tract, using Amapá:

	published	rebuilt
rows	1,492	1,492
unique tract codes	1,492	1,492
geometries with identical area	—	1,492 of 1,492 (100.00%)
total area	142,582.7 km ²	142,582.7 km ² (+0.00%)
attribute variables reproducing exactly	—	24 of 24

The per-variable check is committed as `data-raw/sc_assets_build_verification.csv`.

Two findings from the reconstruction.

First, the codes recorded in `tracts_variables_ref` (V00005, V01007, V06001) could not be found in the IBGE tract aggregates published on the FTP server, and V06001 and V06004 appear in no 2022 file at all. This raised the possibility that the assets mixed census editions, but they do not. The codes refer to censobr (Pereira and Barbosa, 2026), which repackages the same aggregates and prefixes each variable with its source table, so V00005 arrives as `domicilio01_V00005`, except in `ResponsavelRenda` where the codes are bare. The `table_ibge` column of our own reference table corresponds to censobr datasets rather than IBGE files, which was documented nowhere.

Second, geobr returns some tracts as several rows, one per disjoint part, carrying identical attributes. This is not rare: 7 such rows for Amapá, 1,026 for Pará, 1,921 for Amazonas. Left as they are, each row enters the spatial join separately and counts that tract’s CNEFE points two or three times. The published assets dissolve them: for tract 160010505000008 the published geometry measures 490.313 km², the union of its two parts at 91.273 and 399.041 km². The reconstructed script now dissolves them explicitly and explains why.

Neither point was documented in the package before, and both are now recorded in the script and in the manuscript.

Checksums.

`data-raw/sc_assets_checksums.csv` records the SHA-256 and size of every published asset, 27 files and about 1.22 GB per release, against the release tag they are served under. A user can verify that the file they received is the file we shipped. We also noted, in the code that resolves release tags, the constraint this creates: a published edition’s tag must never be renamed, because the checksums are computed against it.

New release.

Once the script was verified, we rebuilt all 27 states with it and published the result as a new release, `sc-assets-v3`, which the package now uses. Compared tract by tract with the original release (`sc-assets-v2`), it has the same 468,097 tracts in all 27 states, and all 3,144,868 value comparisons are identical, as recorded state by state in `data-raw/sc_assets_regen_verification.csv`. The old release stays published, so earlier versions of the package keep working, and the checksums of both releases are recorded. The files users download are therefore the output of the committed script.

On the targets workflow.

We followed the referee’s alternative, “or at least documenting the existing asset-generation pipeline more explicitly”. Rebuilding asset generation as a reproducible `targets` pipeline modelled on `geobr` is the right long-term design and we intend to move that way. It is, however, a larger piece of infrastructure work than we could complete well within this revision. In the meantime, we believe the verified generation script and the published checksums provide the auditability the referee asks for.

The provenance statement in the manuscript.

Section 3 now ends with a “Data provenance” subsection. It names the scripts that build the index and the census tract assets, says where the geometries and attributes come from (`geobr` and `censobr`, now cited), explains the dissolving of multipart tracts, and gives the release tag, the verification result and where the checksums are. Table 3 now says that its variable codes are the ones used by `censobr`, rather than calling them IBGE codes, and `?tracts_variables_ref` says the same.

R1.10: Memory Inefficiency in Pure-R Fallbacks

Memory Inefficiency in Pure-R Fallbacks: In the pure R backends (e.g., within `.cnefe_counts_user_poly_r` and `.cnefe_counts_h3`), the code calls `as.data.frame(tab)` directly on the imported Arrow table before applying `dplyr::filter()` and `dplyr::select()`. For massive datasets like São Paulo (5.7 million records), materialising the entire dataset in memory before filtering completely negates the memory advantages of reading with Arrow. The authors should apply the `dplyr` verbs directly to the Arrow Table and call `dplyr::collect()` as the final step before converting to `sf`.

Response: Accepted and implemented.

The four pure-R backends now apply `transmute()` and `filter()` to the Arrow table and call `collect()` last, as the referee suggests.

We measured memory use before and after the change:

	rows	before	after	
Lauro de Freitas-BA	111k	49.6 MB	47.1 MB	
Fortaleza-CE	1.19M	108.4 MB	68.3 MB	-37%

The saving scales with row count and costs roughly 0.2s, the usual trade of time for memory. On a small municipality it is not measurable, which is expected.

One technical note on the size of the gain. `read_cnefe()` returns an in-memory `arrow::Table`, so the Arrow data is already resident before any backend touches it. What `as.data.frame()` added was a second full copy, as an R data frame of all 34 columns, of which three were then kept. That second copy is what the change removes, and it is the larger of the two.

A larger saving would require `read_cnefe()` to return a lazy Arrow dataset instead of a materialised table. We discuss that option under R1.11.

Two other `as.data.frame()` calls remain on purpose. `read_cnefe(output = "sf")` and `cnefe_export()` are both asked for every column, so there is nothing to push down. Both now carry a comment saying so, since the pattern otherwise looks like the one just fixed.

The regression test checks that the filter is translated to Arrow, not only that the output is correct. When Arrow cannot translate an expression it falls back to R, which would restore the old memory profile while leaving every result identical, so a test that only compared values would not detect it.

R1.11: Architecture Consistency & Efficiency in `read_cnefe()`

Architecture Consistency & Efficiency in `read_cnefe()`: The current implementation of `read_cnefe()` extracts the ZIP file to a temporary directory using `utils::unzip()` before reading it with `arrow::read_delim_arrow()`. This “unpack-every-time” approach is somewhat wasteful in terms of disk I/O and CPU. While using DuckDB’s `zipfs` for zero-extraction streaming is a valid alternative, reading directly from ZIP archives can be computationally expensive for repeated queries. Empirical benchmarks using the Salvador dataset (1.3M rows) show that reading from standard ZIP archives via `zipfs` is significantly slower than native alternatives. Specifically, Gzip was **2.2x faster** than ZIP, and raw CSV was **4.7x faster**. The authors might consider re-compressing the cached data into `.gz` (Gzip) format. DuckDB supports Gzip decompression natively in its core, which avoids the protocol overhead of the community `zipfs` extension. Beyond performance, the authors might also consider the potential stability risks of relying on community-maintained DuckDB extensions in an R package. Some community extensions (e.g., `onager`) are known to encounter issues on macOS due to lack of code-signing and GateKeeper whitelisting. This can cause significant loading delays (4-5 seconds) as the OS re-verifies the extension on every load. While it is unclear if `zipfs` suffers from this specific issue, minimising the surface area of external community dependencies is generally considered a safer practice for production-grade R packages when native alternatives (like Gzip or Parquet) are available. Another resilient option would be to provide a conversion/reorganisation function that allows users to move data from the transient cache to a persistent, optimised structure (like raw CSV or Parquet) on a disk of their choice (see the `spanishoddata` package for an example of this design). Moving toward a lazy DuckDB/DuckSpatial relation (potentially via `duckspatial_df`) instead of a materialised `arrow::Table` would complete this transition to a modern, memory-efficient architecture. I have included three small supplementary scripts: `benchmark_gz_vs_zip.R`, which benchmarks ZIP/`zipfs` reads against native Gzip and raw CSV on the Salvador data; and `suggested_duckdb_impl.R` and `test_duckdb_zipfs.R`, which demonstrate the basic DuckDB/`zipfs` approach. They are not proposed as drop-in production code, but they may help clarify the implementation pattern and provide a starting point for more formal package tests or benchmarks.

Response: Three of the four suggestions are implemented; we did not adopt the lazy relation, for the reasons given below.

We address the four suggestions in turn.

Re-compressing the cache to Gzip: implemented.

We reran the benchmark and added one term that affects the decision: the one-time cost of converting the published ZIP into whatever we cache instead.

Our run, using the referee’s `benchmark_gz_vs_zip.R` as the starting point, with DuckDB reading the full file, median of three repetitions:

	zip	gz	csv	conversion	break-even
Lauro de Freitas-BA	1.25s	1.21s	0.65s	0.52s	15.3 reads
Fortaleza-CE	5.45s	2.38s	1.12s	14.32s	4.7 reads

The referee’s results reproduce closely: 2.29x for gzip against the reported 2.2x, and 4.86x for raw CSV against 4.7x.

Two results decided the choice. Gzip needs no extra disk space, 27.6 MB against the ZIP’s 27.6 MB, whereas raw CSV needs 184.1 MB, 6.7 times more, which is why we did not adopt the fastest option. And the conversion cost is recovered after roughly five reads on a mid-size municipality.

The cache now holds a gzipped CSV. The conversion runs once, on first download, streamed in chunks so peak memory does not scale with the file, and writes through a temporary name so an interrupted run cannot leave something that later passes for a valid cache entry.

Reducing reliance on community extensions: implemented.

The referee’s point about community DuckDB extensions, including macOS code-signing delays of 4 to 5 seconds per load, was an important consideration for us. That cost is paid per connection, and the package opens a connection in every function.

Because DuckDB decompresses gzip natively, `zipfs` is no longer loaded on any connection. It is loaded on demand, only when the file at hand is a ZIP, which now happens only for a cache written by an earlier version or a file the user supplies directly. We could not measure the macOS delay on our platform, so we cannot confirm that figure ourselves.

A conversion function to a persistent store: implemented.

New `cnefe_export(code_muni, path, format)` writes a municipality where the user wants it, as Parquet (the default), CSV or gzipped CSV. `read_cnefe()` gains a `file` argument that reads any of those, and also the raw ZIP as IBGE publishes it, skipping retrieval entirely.

This follows `spanishoddata`, which the referee cites. It also answers Referee 2’s R2.7, which asks whether the package could ingest a locally obtained file if access policy changed. Both paths share the same output code, so they return the same object for the same data.

A lazy DuckDB relation: not adopted.

This is the one suggestion we did not take. A lazy relation, `tbl(con, ...)` in the referee’s `suggested_duckdb_impl.R`, is valid only while its connection is alive. Returning one from `read_cnefe()` requires the connection to outlive the call, with ownership transferred to the returned object and released by a finalizer or an explicit close.

This would conflict with the connection handling requested in R1.12 and R2.C3, which identify connections whose cleanup was not deterministic; R1.12 recommends `withr::defer()`. We adopted it, and every connection now registers a guarded cleanup on its caller’s frame immediately after `dbConnect()` returns. Returning a lazy relation would bring back connections that outlive the function call.

It is also a breaking change to the return type. `read_cnefe()` currently returns an `arrow::Table` or an `sf` object, and a lazy relation is a third type with different semantics, which four internal code paths and two published articles assume.

We have tried to address the memory-efficiency goal behind the suggestion through the parts we did adopt: the export function moves users to Parquet, which is columnar and read lazily, and the pure-R backends now push their filters down to Arrow before collecting, addressed under R1.10.

In the manuscript, Section 3 describes the gzipped cache and the export path, Section 4.5 shows `cnefe_export()` and `read_cnefe(file = ...)` in use, and Section 6 explains to readers why `read_cnefe()` returns a materialised object and how to query a very large municipality lazily from an exported Parquet file.

R1.12: Resource Cleanup and Connection Leaks

Resource Cleanup and Connection Leaks: In several DuckDB backend functions (e.g., `.cnefe_counts_h3`, `compute_lumi`, `tracts_to_h3`), the connection `con` is initialized to `NULL`, followed by a large `utils::capture.output` block that performs the `DBI::dbConnect` and subsequent queries. The `on.exit(DBI::dbDisconnect(con, shutdown = TRUE))` call is only registered *after* this block completes. If an error occurs inside the block—such as an extension loading failure, a memory-intensive SQL error, or a manual user interruption—the function exits before the cleanup is registered, leaving “zombie” DuckDB processes and file handles active. The authors should move the `on.exit()` registration to immediately follow the `dbConnect()` call (even if it’s inside the `capture.output` block) to ensure connections are always closed. Alternatively, the authors could adopt the `withr::defer()` pattern for cleaner resource management. Additionally, wrapping the cleanup in a safety check like `if (!is.null(con) && DBI::dbIsValid(con))` is a defensive best practice to prevent secondary errors during cleanup if the connection failed to initialize.

Response: Accepted. Cleanup is now registered right after connecting, using `withr::defer()`.

The referee correctly describes the failure mode. `on.exit()` was registered only after the whole `capture.output` block had run, so an extension install failure, a SQL error or a user interrupt exited before any cleanup was in place.

We checked every connection site, not only the ones named. Of the six, four registered `on.exit()` between 20 and 56 lines after `dbConnect()`, outside the block, one registered none at all and disconnected manually at the end of the function, and one was acceptable.

All six now route through a single internal helper that opens the connection and registers cleanup on the caller’s frame immediately, before any extension load can fail. Following the referee’s alternative suggestion, we used `withr::defer()`, which made the helper considerably cleaner than manipulating `on.exit()` across frames, and we added `withr` to Imports for it. The handler is guarded with `!is.null(con) && DBI::dbIsValid(con)`, the defensive check the referee recommends.

We verified that the connection is closed both when the caller returns normally and when it aborts mid-block.

R1.13: Test Coverage & Quality

Test Coverage & Quality: While the package includes a `testthat` suite with good use of mocks for offline testing of `read_cnefe()`, the overall package coverage is relatively low at ~25%. Specifically, critical spatial transformation functions such as `tracts_to_h3()` and `tracts_to_polygon()` currently have 0% and ~10% coverage, respectively. Given the complexity of the geospatial logic and the dual-backend architecture, I strongly encourage the authors to expand the test suite to ensure the reliability of these core components.

Response: Accepted. Both functions now have output tests; the CRAN-default figure for `tracts_to_h3()` stays at 0% for the reason given below.

We added tests, and coverage rose for the package as a whole and for `tracts_to_polygon()` in particular. The 0% for `tracts_to_h3()` has a specific cause, explained below.

Coverage depends on how it’s measured. Under CRAN’s defaults, tests marked with `skip_on_cran()` don’t run, and that is the measurement behind the referee’s figures. With

NOT_CRAN=true, which is how the tests run during development and in continuous integration on GitHub, every test runs. Before and after this revision:

	CRAN defaults	Full run (NOT_CRAN=true)
Whole package	38.3% → 46.3%	58.3% → 69.2%
R/tracts_to_polygon.R	10.8% → 10.8%	10.8% → 83.3%
R/tracts_to_h3.R	0.0% → 0.0%	90.0% → 90.0%
R/utils-internal.R	34.5% → 51.9%	58.2% → 60.4%

`tracts_to_polygon()` used to have tests for input validation only. It now has tests that check its output, built so the expected values are exact: a polygon covering a tract receives exactly the tract’s allocated total, a tract split between two polygons is divided correctly, the caller’s columns and CRS are kept, `crs_output` works, and the two-stage diagnostics are reported. They’re described in more detail under R2.C8.

`tracts_to_h3()` already had an end-to-end test of its output, which covers 90% of the file. The referee’s 0% is what CRAN’s defaults produce, because that test calls `skip_on_cran()`. Checking this for the present response, we found that the test was also being skipped in our own continuous integration, on all four machines, because it still required the DuckDB `zipfs` extension, which the package stopped needing when the cache became a gzipped CSV (R1.11). We removed that requirement, and the test, along with one other that had the same problem, now runs on every machine.

The logic of both functions now also runs on CRAN. The deduplication Referee 2 asked for in R2.C1 moved the allocation rules and the diagnostics out of `tracts_to_h3()` and `tracts_to_polygon()` into shared internal helpers (`.build_alloc_sql()`, `.build_interp_diagnostics()` and `.report_interp_diagnostics()`). These helpers need only core DuckDB, with no extension, so we test them directly and without `skip_on_cran()`. That is where the rise in `R/utils-internal.R` under CRAN’s defaults comes from, and it means the calculations themselves are now tested on CRAN, even though the two files still show the figures above.

We kept `skip_on_cran()` on the end-to-end tests because of a crash. It was added in version 0.2.1 after the DuckDB spatial extension caused a segfault on CRAN’s `r-devel-linux-x86_64-fedora-clang` check machine, an incompatibility between the clang-built DuckDB binary and the GCC-built extension (duckdb/duckdb-r#1107). Removing it would risk the package failing on CRAN again, so we have kept it, even though the CRAN-default figure for that file stays at 0%.

Referee 2

General

The article presents **cnefetools**, an R package that provides convenient access to Brazil’s National Address File for Statistical Purposes (CNEFE) — a geocoded register of approximately 110 million addresses from the 2022 Brazilian Census published by IBGE. The package offers four capabilities beyond simple data access: (1) downloading and caching CNEFE data for any of Brazil’s 5,570 municipalities; (2) spatial aggregation of address counts onto H3 hexagonal grids or user-supplied polygons; (3) computation of land-use mix indices (EI, HHI, BAL, ICE, HHI_adp,

BGBI) at the same spatial units; and (4) dasymetric interpolation of census tract variables (population, demographics, income) to H3 grids or custom polygons using CNEFE dwelling points as ancillary data. Computationally intensive operations leverage a DuckDB backend with H3 and spatial extensions, achieving up to 13× speedup over a pure-R fallback, with the pure-R backend available for restricted environments.

This package is important to the R community. CNEFE data is distributed as per-municipality semicolon-delimited CSVs inside ZIP files on an FTP server with no unified programmatic interface, making it cumbersome to work with. `cnefetools` dramatically lowers the barrier to entry. The choice of DuckDB + Arrow + H3 is modern and well-suited to the data scale. The dasymetric interpolation functionality fills a real methodological gap — no other CRAN package implements land-use mix indices, and existing interpolation packages (`sf`, `areal`, `populR`, `smile`) either use weaker areal-weighting, lack ancillary data, or use building footprints rather than dwelling counts. The article’s comparison with these alternatives (Section 4.4) is well done.

The article is clearly written and well-structured. Section 2 provides excellent background on the CNEFE dataset, including its production process, quality metrics (MSE of 5.84m under ideal conditions), and distribution challenges. Section 3 clearly explains the three key design decisions (pre-built URL index, persistent cache, dual backend). The MAUP discussion in Section 4.3 and the ecological fallacy caveat in Section 4.4 show methodological awareness. The diagnostic reporting system for dasymetric interpolation is a thoughtful feature.

Shortcomings are relatively minor. The main ones are: (1) substantial code duplication between `cnefe_counts.R`, `compute_lumi.R`, and the `tracts_to_*` functions; (2) aggressive suppression of DuckDB output via nested `capture.output()` that can mask real errors (acknowledged in open GitHub issue #57); (3) the `compute_lumi()` function hardcodes a binary residential/non-residential split, acknowledged in Section 6 but still a significant limitation; and (4) some edge cases in input validation and inconsistent return behavior.

Response: Thank you for this summary. The four shortcomings it lists are answered below, under R2.C1 (duplication), R2.C2 (`capture.output`), R2.1 (the binary split) and R2.C4 to R2.C6 (edge cases and return types).

Detailed comments

R2.1: Binary land-use classification in `compute_lumi()`

Binary land-use classification in `compute_lumi()`. The LUMI indices treat `COD_ESPECIE == 1` as “residential” and everything else (2–6, 8, excluding 7) as “non-residential.” The article acknowledges this limitation in Section 6 and notes plans to expand to multi-category formulations. However, the article could strengthen the discussion by noting specific use cases where the binary split is inadequate — for example, distinguishing commercial from institutional land use matters for urban health studies, and agricultural establishments (type 3) are qualitatively different from urban non-residential types. The article already notes that type 3 (agricultural) accounts for 3.7% of addresses and type 6 (other) for 10.5% — collapsing these very different categories into one “non-residential” bucket discards substantial information.

Response: Accepted. Section 6 now discusses the cases the referee mentions.

It explains that the binary split places agricultural establishments (type 3, 3.7% of addresses), which are qualitatively different from urban uses, in the same group as schools and hospitals, and that it merges other-purpose establishments (type 6), a heterogeneous category that includes commerce and services and makes up 10.5% of addresses, with educational, health and religious ones. It

includes the referee’s example of an urban health study that needs to separate commercial from institutional uses and can’t do so with these indices. It also points out that `cnefe_counts()` keeps all eight address types, so users who need a multi-category index can build one from its output today.

Section 4.3 states what the binary split does and doesn’t measure, as described under R1.1 and R1.3, and multi-category formulations stay on the list of future work.

R2.2: Exclusion of `COD_ESPECIE == 7`

Exclusion of `COD_ESPECIE == 7`. The article explains that type 7 (“under construction or renovation”) is excluded from `compute_lumi()` due to its “transitional nature.” However, with 3.5 million such records (3.2% of all addresses), and likely concentrated in rapidly urbanizing peripheries, their exclusion could systematically bias LUMI indices in precisely those areas where land-use classification matters most. The article should at minimum discuss the direction and likely magnitude of this bias. Additionally, `cnefe_counts()` does NOT exclude type 7 (it appears in `addr_type7`), creating an inconsistency between the two functions that should be documented.

Response: Accepted for the documentation and the direction of the bias; we have not quantified its magnitude. Checking the exclusion also revealed a defect, now fixed.

The different treatment of type 7 is now documented in both functions. `?compute_lumi` states that type 7 is excluded and why, `?cnefe_counts` states that it is retained and reported as `addr_type7`, and the two help pages cross-reference each other. The difference is intentional, since a building under construction describes a transitional state and not a realised land use, but it was not documented.

On the direction of the bias, Section 4.3 now has a paragraph on it. Such buildings are likely to be concentrated where a city is expanding, so leaving them out makes the indices describe those areas as they are now rather than as they will be once the buildings are occupied. If the buildings end up in the same proportions as existing addresses, among which private households are the large majority, the indices understate the residential share those areas are heading towards, and the effect is largest where type 7 makes up a large share of addresses. The same paragraph notes that `cnefe_counts()` keeps these addresses as `addr_type7`, so users can check how many there are in their own area of interest.

On the magnitude, we have not quantified it. Estimating how the exclusion shifts the indices across rapidly urbanising peripheries would require a new multi-municipality analysis, which is beyond the scope of a software paper, and a figure from a single municipality could be misleading, so the manuscript states the direction only.

While checking that the exclusion was applied consistently, we found that it was not. Of the four code paths in `compute_lumi()`, the DuckDB backend for user-supplied polygons never applied the filter, so every index it returned was computed over a denominator that still contained type 7. Since `backend = "duckdb"` is the default, this was the path most users of `polygon_type = "user"` would have hit. Cross-checked against `cnefe_counts()` on the 19 neighbourhoods of Lauro de Freitas, the discrepancy reached 0.13 in `p_res` and 0.32 in `bgb_i`, in a municipality where type 7 is 3.7% of addresses. The fix is one clause of SQL, the two backends now agree to zero difference on all seven indices, and the defect and its evidence are recorded in the package issue tracker.

R2.3: Global P in BGBI: scope and computation

Global P in BGBI: scope and computation. The article states that “The citywide residential share P is computed from the full municipality CNEFE” (Section 4.3), which is correct and appropriate. However, two subtleties deserve mention: (a) P is derived from CNEFE dwelling counts, not from the population census, so it reflects the *distribution of address types* rather than the *distribution of residents* — a municipality with many small households and few large institutions would have a high P regardless of population; and (b) when `polygon_type = "user"`, P still reflects the entire municipality, not just the area covered by the user’s polygons, which means the BGBI baseline is externally defined rather than locally computed.

Response: Both points are now documented; the computation is unchanged.

On (a), P is derived from CNEFE address-type counts rather than from census population, so it describes the distribution of address types and not the distribution of residents. This is now stated explicitly in `?compute_lumi` and in Section 4.3.

On (b), P does remain citywide under `polygon_type = "user"`, and this is intentional. P describes the context in which the addresses sit, which is the municipality. A user-supplied sub-area, even when it covers only part of the city, is still part of that wider context, so a citywide baseline is the intended reference. A baseline recomputed over the covered area would measure a different quantity, namely mix relative to the sub-area itself instead of the city, which is the comparison the BGBI was designed to avoid. This reasoning is now explained in both the documentation and the manuscript.

We also note that P is used by two indices, the BGBI and the Balance Index, where it enters through $r = P/(1 - P)$. The other indices are computed entirely within each spatial unit.

R2.4: Dasymetric interpolation: unallocated population and ecological fallacy

Dasymetric interpolation: unallocated population and ecological fallacy. The article provides an excellent discussion of the ecological fallacy (Robinson, 1950) and the limitations of the proportional-dwelling assumption (Section 4.4). The diagnostic output (shown in the article) quantifies unallocated totals per variable. One enhancement would be to discuss typical magnitudes: for the Lauro de Freitas example, the unallocated `pop_ph` is 0.00% — but what about larger, more complex municipalities like São Paulo? Reporting typical unallocation rates across a sample of municipalities would help users calibrate expectations. Also, the diagnostic is always emitted regardless of `verbose`, which is a good design choice.

Response: We measured São Paulo: the unallocated share is 0.76%.

We ran `tracts_to_h3()` on São Paulo (3550308) with the same variables and resolution as the Fortaleza example in the `tracts_to` article, so the two diagnostics are directly comparable. Unallocated `pop_ph` is 86,998 of 11,394,071, or 0.76%. Unmatched CNEFE points are 2,935 of 4,996,529, or 0.06%, and 100% of allocated points map to H3 cells.

This matches the figure reported in the package’s `tracts_to` article, on the website, for `tracts_to_polygon()` on São Paulo traffic zones. This is expected: unallocation occurs in stage 1, in the match between tracts and CNEFE points, so it does not depend on the target geometry.

Across the three worked examples the figures are: Lauro de Freitas 0.00%, Fortaleza 0.09%, São Paulo 0.76%. It stays below 1% even in São Paulo, the largest and most complex municipality in the country.

We have not added a sample across municipalities. A published average would have to be recomputed for every future CNEFE edition and would still not tell a user what happens in their own municipality. The package instead reports the unallocated total on every call, regardless of `verbose`, as the referee notes. This gives each user the figure for their own input, which we think is more useful than an average.

R2.5: Single-municipality scope

Single-municipality scope. The article acknowledges in Section 6 that “core analytical workflows are organised around municipality-specific inputs, which limits analyses that span multiple municipalities.” This is a significant practical limitation. Many research questions require cross-municipality analysis (e.g., metropolitan areas spanning multiple municipalities). The article’s discussion of plans to address this is appreciated, but the current constraint deserves slightly more prominence — perhaps a note in Section 3 about how users could currently work around it (calling functions in a loop and combining results, with appropriate caveats about the H3 grid being municipality-specific).

Response: Accepted. Sections 3 and 6 now give the constraint more prominence and describe the workaround.

The `tracts_to` article already had a section on scaling to multiple municipalities, pointing users at `purrr::map()` over municipalities followed by `dplyr::bind_rows()`. We have added a pointer there to a published worked example applying that iterative workflow across the 39 municipalities of the São Paulo metropolitan region: <https://pedreirajr.github.io/website/posts/2026-02-18-census-tracts-rmsp/>

We agree with the referee that the constraint deserves more prominence. Section 3 now states that every function works on one municipality at a time and points to Section 6, where the limitation is discussed. Section 6 now explains how analyses that span several municipalities can be done today, citing the worked example, and gives two caveats: the H3 grid is built from each municipality’s own boundary, so a cell on a shared border belongs to only one of them, and the citywide share P is computed per municipality.

R2.6: No backend parameter for `tracts_to_*`

No backend parameter for `tracts_to_*`. The article states that `tracts_to_h3()` and `tracts_to_polygon()` “do not expose this argument and rely on DuckDB exclusively” (Section 3). This means these functions have a hard dependency on DuckDB, duckspatial, and the spatial/H3 extensions — including the community extensions (zipfs, h3) which may fail to install on some platforms. Given that the article benchmarks show the pure-R backend is only $\sim 13\times$ slower (and comparable for small municipalities), the absence of a fallback seems unnecessarily restrictive. The article should state whether a pure-R fallback is planned or intentionally excluded, and why.

Response: The omission is intentional, and the manuscript now explains why.

We had not stated our position before. It is now explained in Section 3 of the manuscript and in the roxygen of both functions.

We think the benchmark is less informative for these functions than it may appear, because the speed-up figure comes from `cnefe_counts()`, which is a tabular aggregation over address records.

The `tracts_to_*`() functions are dominated by a different workload, a spatial overlay between the full CNEFE point set of the municipality and the census tract polygons, followed by a second overlay onto the target units. That cost doesn't scale like the tabular case, and it's the step where DuckDB's spatial index does most of the work. We haven't built a pure-R version to time, so we don't put a number on it, but we'd expect the gap to be much wider than the one measured on the tabular aggregation, and a fallback that is impractical on the large municipalities where users would most need it would add maintenance cost for little benefit. Section 3 now gives this reason.

R2.7: Data-access risk: availability vs. policy

Data-access risk: availability vs. policy. Section 6 frames the CNEFE dependency purely as a *technical* risk (“data access depends on the availability and stability of the IBGE FTP server... this does not represent a fundamental barrier in practice, since the CNEFE is open data, freely available to the public”). This overlooks a distinct and less easily mitigated risk: that access *policy* itself could change. CNEFE is unusually granular for an openly published national statistical product — most national statistical offices (including, under GDPR, most EU members) release geocoded address data only as aggregated grids precisely to avoid re-identification concerns, whereas CNEFE publishes individual geocoded points nationwide. The authors should distinguish “server unavailable” from “access restricted” as separate risk categories, and could note how the package’s architecture would (or would not) accommodate a future authenticated/restricted-access retrieval path — e.g., whether `read_cnefe()` could ingest a locally-obtained file independent of the FTP download step.

Response: Accepted. Section 6 now treats availability and access policy as separate risks.

Section 6 treated the dependency as a question of server uptime and considered it minor because CNEFE is open data. That reasoning only covers availability, not the case the referee raises, where the data remains published but the terms of access change. We now separate the two.

- **Availability.** The IBGE FTP server may be unreachable. Mitigated by caching, by retries, and by the fact that a cached municipality file keeps working offline.
- **Access policy.** Publication of individual geocoded address points nationwide is unusually granular for a national statistical product, and, as the referee notes, most offices, including EU members under GDPR, publish only aggregated grids. A future re-identification review could move CNEFE behind authentication, aggregation, or licensing. Caching does not mitigate this, and neither does anything else the package can do from its own code.

On architecture, the referee’s example is now possible. `read_cnefe()` has a `file` argument, added under R1.11, that reads a locally obtained file, including the ZIP as IBGE publishes it, without going through the FTP download. Section 6 says so, and also notes a limit: the analytical functions (`cnefe_counts()`, `compute_lumi()` and `tracts_to_*`()) still get their data through the cache, so they would need the same option to keep working under restricted access.

R2.8: Benchmark completeness

Benchmark completeness. The benchmarks (Section 5) are well designed, varying both municipality size and H3 resolution. However: (a) they only benchmark `cnefe_counts()`, not `compute_lumi()` or `tracts_to_*`() — users would benefit from knowing the performance profile

of all heavy functions; (b) they report only elapsed time, not peak memory usage. For São Paulo (5.7M addresses, 901 MB uncompressed), the in-memory DuckDB approach may require several GB of RAM. Reporting peak memory would help users anticipate hardware requirements; (c) the benchmark was run on a high-end machine (14-core i9, 32 GB RAM) — reporting results on a more modest system would be more representative of typical researcher hardware.

Response: We answer (a), (b) and (c) separately below, as R2.8a, R2.8b and R2.8c. Answering them meant rerunning the benchmark, and two changes since submission affect the numbers already in Section 5, not only the new ones.

First, the package itself changed during this revision. The cache is now a gzipped CSV (R1.11), and the pure-R backend now pushes its filters down to Arrow before collecting (R1.10). Second, the measurement protocol changed. The submitted figures came from a single run in which every DuckDB case preceded every pure-R case. On our machine the same call on the same input drifts by up to a factor of two within one session as the CPU changes thermal and frequency state, so timing the backends in blocks loads that drift onto one of them. The new harness discards a warm-up, visits every configuration once per pass in rotation, repeats each pass three to five times in a fresh R session each time, and reports the median with the observed range. Every replicate is committed in `data-raw/bench_r2_8.csv` in the package repository. The speedups in the revised Section 5 therefore differ from the submitted ones for both reasons.

R2.8a: Benchmark the other heavy functions

Response: Accepted. All four heavy functions are now benchmarked.

`compute_lumi()` performs the same as `cnefe_counts()`. The ratio between them stays at 1 across a 28-fold range of input size, on both backends, with no trend:

H3 res 8, median seconds	<code>cnefe_counts()</code>	<code>compute_lumi()</code>	ratio
Vitória da Conquista-BA, DuckDB	3.79	4.50	1.19
Curitiba-PR, DuckDB	3.53	3.48	0.99
São Paulo-SP, DuckDB	5.58	5.58	1.00
Vitória da Conquista-BA, pure R	4.69	4.45	0.95
Curitiba-PR, pure R	9.19	9.26	1.01
São Paulo-SP, pure R	93.25	90.61	0.97

The one ratio away from 1, Vitória da Conquista on DuckDB, sits inside that configuration’s own replicate dispersion, which is 31% and 10% respectively at a scale where fixed startup cost dominates the measurement.

This is expected, because the two functions share the same DuckDB scan over the same gzipped CSV, the same `h3_latlng_to_cell()` assignment and the same `GROUP BY`. `compute_lumi()` then aggregates to two columns instead of eight, so it skips a pivot, and adds the land-use mix arithmetic.

That arithmetic runs once per hexagon, not once per address, which on São Paulo is roughly four orders of magnitude fewer operations. For this reason we report the equivalence instead of a separate figure for `compute_lumi()`. The per-replicate measurements behind this table are committed in `data-raw/bench_r2_8.csv`.

The `tracts_to_*`() functions do need their own figures. They add a point-in-polygon overlay of the full CNEFE point set against census tract polygons, plus a Parquet download of the tract attributes, and they are DuckDB-only with no `backend` argument.

H3 res 9, DuckDB	<code>tracts_to_h3()</code>	<code>tracts_to_polygon()</code>
Vitória da Conquista-BA (~200k)	9.77 s, 0.66 GB	7.27 s, 0.80 GB
Curitiba-PR (~900k)	6.91 s, 0.61 GB	5.25 s, 1.09 GB
São Paulo-SP (~5.7M)	15.65 s, 1.93 GB	15.53 s, 4.28 GB

Two points stand out. The dasymetric functions do not scale with address count the way `cnefe_counts()` does: Vitória da Conquista is slower than Curitiba despite being four and a half times smaller, because at that size the cost is dominated by loading the state’s census tract Parquet, and Bahia’s is more than twice the size of Paraná’s. And `tracts_to_polygon()` is the most memory-hungry path in the package, at 4.28 GB on São Paulo, because an overlay against arbitrary user polygons cannot use the H3 shortcut.

R2.8b: Report peak memory

Response: Accepted. Peak memory is now reported, and it is highest for the pure-R backend.

Peak memory is now reported for every configuration. We measure the operating system’s peak working set for the whole R process, through `ps`, instead of R-level allocation tracking such as `bench::mark()`’s `mem_alloc`. This matters because R-level tracking counts only the R heap, while DuckDB allocates in C++ outside it, so it would understate DuckDB’s memory use.

The comment anticipates that “the in-memory DuckDB approach may require several GB of RAM” for São Paulo. The measurements confirm that several GB are needed, but by the pure-R backend:

<code>cnefe_counts()</code> , H3 res 8, peak RSS	DuckDB	Pure R	ratio
Vitória da Conquista-BA (~200k)	0.46 GB	0.70 GB	1.5
Curitiba-PR (~900k)	0.43 GB	1.72 GB	4.0
São Paulo-SP (~5.7M)	0.70 GB	9.03 GB	12.8

DuckDB streams the CSV and aggregates as it goes, so it never materialises the address table and its footprint stays nearly flat, rising only from 0.43 to 0.70 GB across a 28-fold increase in input. The pure-R backend holds the filtered table in R memory, so its footprint scales with the input and reaches 9 GB on São Paulo.

This changes advice the package was giving. Both the manuscript and the article recommended the pure-R backend for constrained environments. That is right when the constraint is that DuckDB extensions cannot be installed, which is the case it was written for, and wrong when the constraint is memory. The article now separates the two, and so does Section 5 of the manuscript.

Two further points on anticipating hardware requirements. DuckDB sets `memory_limit` to 80% of installed RAM, so it asks for less on a smaller machine, and exceeding that limit makes it spill to a temporary directory rather than fail: a small machine is slower, not blocked. And we have added a `cnefetools.duckdb_config` option, so the budget can now be set explicitly rather than only inferred from the machine:

```
options(cnefetools.duckdb_config = list(threads = 4, memory_limit = "4GB"))
```

R2.8c: Report on more modest hardware

Response: Partly accepted. We report runs with limited threads and memory on our machine, and state what they do not show.

We limited DuckDB to four threads and 4 GB and reran the largest municipality:

São Paulo-SP (~5.7M addresses)	full machine	4 threads, 4 GB
<code>cnefe_counts()</code>	5.58 s, 0.70 GB	7.01 s, 0.45 GB
<code>tracts_to_h3()</code>	15.65 s, 1.93 GB	20.93 s, 1.75 GB

The largest municipality in the dataset aggregates in about seven seconds within a 4 GB budget, 26% slower than on the full machine while using 36% less memory. The dasymetric interpolation takes 34% longer under the same constraint. Neither degrades sharply.

This shows that the package works within a small resource budget. It does not show performance on older hardware: the storage, the RAM speed, the CPU generation and the operating system are all still ours. An older laptop would do worse, particularly on the initial read, which is I/O-bound and which no amount of thread limiting reproduces. The manuscript states this limitation and does not present the constrained run as a substitute for a second machine. The machine specification is reported so that readers can scale expectations, as the referee suggests.

One methodological note, since it is visible in the committed data. Our first attempt also pinned CPU affinity to four specific processors, and it produced 79% and 82% replicate spread against roughly 20% elsewhere. The per-replicate machine-load column we now record showed elapsed time tracking background load almost exactly: Windows schedules other work onto the low-numbered processors by preference, so the pinned runs were competing for the busiest cores instead of simply running with less parallelism. Limiting DuckDB’s thread count alone gives the intended budget without that artefact, and brought the spread to 10% and 8%. We mention it in case it is useful to others running similar measurements.

R2.9: Address count inconsistency with article

Address count inconsistency with article. The abstract states “approximately 110 million valid addresses,” while Section 2 reports “111.1 million geographic records, of which 98.95% were validated as spatially coherent” and “more than 110.7 million records (99.68%) were geocoded at the address level.” The abstract should more precisely say “approximately 111 million” or clarify what “valid” means (all records? only geocoded ones?).

Response: Accepted. The abstract, the Introduction and Section 2 now describe the figure correctly.

We verified every figure against the IBGE methodological note that documents this release, *Censo Demográfico 2022: Coordenadas Geográficas dos Endereços. Nota metodológica n. 01* (IBGE, 2024). The figures in Section 2 of our manuscript are all correct. The total is 111,102,875, of which 109,936,532 (98.95%) had their coordinates validated, and 110,744,837 (99.68%) are geocoded at address level, splitting into 103,943,086 original, 6,054,473 modified and 747,278 estimated. The species breakdown in Section 2 also matches the note’s Annex 3 exactly.

The abstract, however, had a further problem beyond the undefined word “valid”. IBGE does not count addresses in this release. It counts *espécies de endereço*, address species, and the note is explicit that the two are not the same quantity: “since each address may contain more than one species at the same time [...] the published coordinates add up to a number greater than the total number of addresses” (our translation from the Portuguese). One address can host several species, so the 111.1 million figure exceeds the number of addresses, and the note defers the address total itself to a future CNEFE release. It has not been published.

Our abstract therefore describes 110 million as “valid addresses” when the underlying quantity is validated coordinate records for address species, and the true address count is both smaller and, at present, unknown. Section 2 mostly gets this right by saying “records”, but its opening sentence, “Each CNEFE record corresponds to a single address point”, made the same slip, and the Introduction repeated the 110 million “addresses”.

For this reason we did not change the figure to “approximately 111 million”, since that total includes the 1.05% whose coordinates were invalidated or never captured, and it is still not a count of addresses.

The abstract and the Introduction now report 111.1 million records, one for each type of use found at an address. Section 2 now opens by explaining that each record is one address species, so an address with more than one use appears more than once and records outnumber addresses, citing the IBGE note, and it says that the rest of the paper uses “address” to mean a CNEFE record.

We thank the referee for this comment, which led to an important correction in the abstract.

R2.10: Citation for BGBI

Citation for BGBI. The BGBI is attributed to “Pedreira Junior et al. (2025)” on engrXiv, which is a preprint server. This should be clearly labeled as a preprint (e.g., “engrXiv preprint”), and if the method has since been peer-reviewed and published, that citation should replace or supplement the preprint reference.

Response: Accepted. The preprint has since been published and the reference is updated.

The BGBI article appeared in peer-reviewed form after submission:

Pedreira Junior, J. U., Louro, T. V., de Assis, L. B. M., Brito, P. L., Bomfim, F. G. (2026). BGBI: A citywide-referenced and bidirectional land use mix index for planning and policy evaluation. *Land Use Policy*, 169, 108135. <https://doi.org/10.1016/j.landusepol.2026.108135>

The manuscript now cites this article for the BGBI, including in the discussion of Figure 3, and the package references in `?compute_lumi` and the README cite it too.

One clarification: only the BGBI was published. The adapted HHI (`hhi_adp`) appears only in the engrXiv preprint, so we keep both references rather than replacing one with the other, and the preprint is now labelled “engrXiv preprint” in the bibliography, as the referee suggests.

R2.11: smile package comparison

smile package comparison. The article mentions the `smile` package in Section 4.4 as a model-based alternative, but does not give a proper citation (no entry in the Bibliography). If this is a published/CRAN package, it should be cited fully; if not, a URL or note would be appropriate.

Response: Accepted. smile is now cited.

The package is on CRAN (version 1.1.0). It now has a bibliography entry and is cited where it’s mentioned in Section 4.4.

Referee 1 raises the same omission in R1.5, alongside `arrow`, `duckspatial` and `piggyback`, and the same change addresses both comments.

R2.12: LazyData: true in DESCRIPTION

LazyData: true in DESCRIPTION. The package sets `LazyData: true` but this is deprecated per CRAN policy and generates a NOTE on recent R CMD check versions. It should be removed.

Response: We have kept LazyData: true, for the reasons below.

We checked this with a full R CMD check on R 4.6.0, which reports `checking LazyData ... OK`, with no NOTE, warning or error in the log. As far as we can tell, the NOTE is raised when `LazyData` is declared without a `data/` directory, or when the lazy-load database is large enough to warrant `LazyDataCompression`. Neither applies here: the package ships one `data/` file, `tracts_variables_ref.rda`, at 716 bytes.

Removing the field would also change how that dataset is accessed. Without `LazyData`, `tracts_variables_ref` is no longer available on `attach` and can only be reached through an explicit `data(tracts_variables_ref)` call. Two documented usages depend on direct access: the `tracts_to` article, which pipes the object straight into `dplyr`, and the example in `?tracts_variables_ref` itself.

We would be glad to revisit this if the referee has a specific R or check version in mind where the NOTE appears.

Code: usability (no action requested)

- **Workflow is clear and user-friendly.** The four main functions (`read_cnefe`, `cnefe_counts`, `compute_lumi`, `tracts_to_h3/tracts_to_polygon`) form a logical pipeline from data access → aggregation → analysis → interpolation. The consistent interface (same `code_muni`, `year`, `cache`, `verbose` parameters across functions) is well designed. The article’s statement that “users call whichever function fits their need

directly, without a separate data-loading step” (Section 3) accurately reflects the package’s self-contained design.

- **Documentation is thorough.** Each exported function has detailed roxygen documentation with `@description`, `@details`, `@param`, `@return`, and `@examples`. The pkgdown site with 6 vignette articles is excellent. The `tracts_variables_ref` data set mapping package variable names to IBGE codes is a very helpful reference that the article appropriately references in Table 3.
- **Coverage diagnostics** in `tracts_to_*`() are informative and well-structured with the two-stage (Tracts → CNEFE → Polygons/H3) reporting, as illustrated in the article’s example output.
- **Cache management** is well-designed: `clear_cache_muni()` and `clear_cache_tracts()` with flexible UF/codes filtering, plus the `cache = FALSE` option for temporary file usage, give users good control over disk space.

Response: Thank you. The package website now has seven articles, including a new one on the cache and on keeping exported copies of the data (see R1.7).

Code quality

R2.C1: Substantial code duplication

Substantial code duplication. The allocation logic (the `alloc_exprs` CASE/WHEN SQL construction) is duplicated nearly verbatim between `tracts_to_h3.R` (lines 334–421) and `tracts_to_polygon.R` (lines 451–538). The DuckDB extension loading pattern (`utils::capture.output(utils::capture.output({...}))`) appears in at least 6 places. The polygon validation logic (checking `inherits(polygon, "sf")`, geometry type, CRS validity) is repeated across `cnefe_counts()`, `compute_lumi()`, and `tracts_to_polygon()`. The diagnostic reporting logic (unallocated totals, NA tracts, no-eligible-dwelling tracts) is also duplicated between the two `tracts_to_*` functions. These should be extracted into shared internal helpers to reduce maintenance burden and the risk of divergence.

Response: Accepted. All four duplicated blocks are now shared internal helpers.

We checked each item against the code. All four were confirmed, and two were more extensive than reported.

Duplication	Reported	Found
Allocation SQL builder	duplicated	byte-identical, 90 lines each, confirmed with <code>diff</code>
DuckDB extension loading	“at least 6 places”	13 nested <code>capture.output</code> sites, plus two local <code>.duckdb_load_ext()</code> closures the report did not name
Polygon validation	duplicated across 3 functions	confirmed, differing only in one error message
Diagnostic reporting	duplicated	confirmed, roughly 200 lines each

Four internal helpers now carry the shared logic: a DuckDB connect-and-quiet helper, the allocation SQL builder, the polygon validator, and the two-stage diagnostics builder and reporter. `tracts_to_h3.R` went from 801 to 425 lines and `tracts_to_polygon.R` from 973 to 580.

Two further points.

First, the diagnostic blocks had already diverged. One ended the “Tracts with NA totals” line with a full stop and the other did not, and one wrapped every query in `suppressMessages()` while the other left them bare. Neither divergence was intentional and neither was visible without diffing the two files, which illustrates the risk the referee describes.

Second, because the test suite was thin (R2.C8), we saved a reference output before changing anything: all four exported functions, across every backend and aggregation mode, with a SHA-256 per normalised result. That reference caught a defect in `compute_lumi()` before the refactor began, reported under R2.2. Every commit in this work is verified byte-identical against it.

R2.C2: `capture.output` masking errors (GitHub issue #57)

`capture.output` masking errors (GitHub issue #57). The double-nested `utils::capture.output(utils::capture.output(expr, type = "message"), type = "output")` pattern is used extensively to suppress DuckDB progress bars. However, this also swallows error messages from DuckDB, making debugging very difficult when something goes wrong inside the SQL. This is acknowledged in open issue #57. Since DuckDB’s config already sets `enable_progress_bar = FALSE` and `print_progress_bar = FALSE` at connection time, the `capture.output` wrapping may be partially redundant. A better approach would be to use `tryCatch` to surface errors while still suppressing routine output, or to rely on the DuckDB config options alone.

Response: Accepted. The nested `capture.output()` is gone, and DuckDB errors now reach the user.

The nested `capture.output(capture.output(expr, type = "message"), type = "output")` installed a message sink around the entire block. As a result, DuckDB failures reached users and CI logs with the step marked as failed but no error text.

The replacement suppresses only what needs to be suppressed, DuckDB’s stdout progress output and its startup messages, without installing a message sink: `suppressMessages()` muffles messages and leaves conditions of class `error` untouched. Errors now propagate with their message intact, which we verified.

Together with the connection fix in R1.12 and R2.C3, this resolves both parts of the issue the referee cites, since the late `on.exit()` registration was a consequence of the same block.

All 13 occurrences of the nested idiom are gone. One remains, inside the helper that replaced them.

R2.C3: DuckDB connection lifecycle inconsistency

DuckDB connection lifecycle inconsistency. In `.compute_lumi_user_poly_duckdb()` (`compute_lumi.R`:869), the connection is disconnected manually with `DBI::dbDisconnect(con, shutdown = TRUE)` outside the `on.exit()` handler. If an error occurs before that line, the connection leaks. Other functions (e.g., `.cnefe_counts_h3()`, `.cnefe_counts_user_poly_duckdb()`) correctly use `on.exit(DBI::dbDisconnect(...), add = TRUE)`. This inconsistency should be fixed by consistently using `on.exit()`.

Response: Accepted.

`.compute_lumi_user_poly_duckdb()` was indeed inconsistent with the rest: it was the only function with no `on.exit()` at all, disconnecting manually at the end. Any error before that line leaked the connection.

It now uses the same internal helper as every other site, which registers a guarded cleanup on the caller's frame immediately after `dbConnect()`.

Referee 1 raises the same problem in R1.12 with a wider scope, and the same change addresses both.

R2.C4: Edge case: zero-feature polygon (GitHub issue #71)

Edge case: zero-feature polygon (GitHub issue #71). When `polygon` has 0 rows, the package produces a cryptic error rather than a clear message. The validation in `cnefe_counts()`, `compute_lumi()`, and `tracts_to_polygon()` checks for `inherits(polygon, "sf")` and geometry type, but not for `nrow(polygon) == 0`. An explicit check with a clear error message should be added.

Response: Accepted. The check is in the shared validator, so it covers all three functions.

The cryptic error came from the following chain: `sf::st_union()` on a zero-row `sf` yields an empty geometry, `sf::st_centroid()` of that an empty POINT, and `sf::st_coordinates()` a zero-row matrix, so the test on `centroid_coords[2]` saw NA.

We put the check in the shared polygon validator introduced for R2.C1, so `cnefe_counts()` and `compute_lumi()` are covered as well as `tracts_to_polygon()`. All three had the same latent failure.

The error now aborts at input validation and names the common cause, since filtering a `geobr` dataset for a municipality it does not cover is how users reach this state. New tests assert the abort in all three functions.

R2.C5: Edge case: read_cnefe() with partially NA coordinates

Edge case: read_cnefe() with partially NA coordinates. The function correctly aborts when *all* coordinates are NA (line 177), but silently drops rows with individual NA coordinates (line 175). A `verbose` message reporting how many rows were dropped would help users notice data quality issues.

Response: Accepted. read_cnefe() now reports how many rows were dropped.

Dropping rows without a usable coordinate is still the right behaviour, since they cannot become geometry, but doing it silently meant a data quality issue in a municipality file went unnoticed unless the user compared row counts before and after.

`read_cnefe(output = "sf")` now reports the count and the percentage under `verbose`, and stays silent when nothing is dropped. The existing abort for the all-missing case is unchanged, since that is a different situation and already had a clear message.

R2.C6: `compute_lumi()` returns NULL for empty results (R backend only)

`compute_lumi()` returns NULL for empty results (R backend only). At `compute_lumi.R:449-450`, the R backend returns NULL when no valid points exist after filtering, while the DuckDB backend returns an empty sf object. This inconsistency could cause downstream errors if callers don't check for NULL. Both backends should return the same type.

Response: Accepted. Both backends now return a zero-row sf object.

Returning NULL was also inconvenient regardless of the backend: `compute_lumi()` returns an object users pipe into `dplyr` and `sf` calls, and NULL forces a defensive check that the documentation never mentioned.

Both sites in the hex path now return a zero-row `sf` carrying the documented columns in CRS 4326, so the shape of the result is stable and only the row count varies. A caller can filter, join or bind the result without a special case.

We also moved the accompanying messages from `message()` to `cli`, which is what the rest of the package uses.

R2.C7: `build_h3_grid()` is documented but marked internal

`build_h3_grid()` is documented but marked internal. It has a .Rd file in `man/` but is marked `@keywords internal`. Since it's not exported (not in `NAMESPACE`), the Rd file is unnecessary clutter. Either export and document it properly (it could be useful for advanced users) or remove the Rd file.

Response: Accepted, resolved by removing the Rd file.

`build_h3_grid()` is not exported, so its manual page was unnecessary. We swapped `@keywords internal` for `@noRd` and deleted `man/build_h3_grid.Rd`.

We considered the other option the referee offers, exporting it for advanced users, and decided against it. Exporting mid-revision would create a stability promise the package would then have to carry indefinitely, for a helper whose signature we may still want to change. For example, `build_h3_grid()` gained a `year` argument during this revision, under R1.8b.

R2.C8: Test coverage could be stronger

Test coverage could be stronger. The tests use a small offline fixture ZIP and mock bindings, which is good for CI reliability. However:

- `test-tracts_to_polygon.R` only tests input validation, not actual interpolation output (unlike `test-test-tracts_to_h3.R` which does test output correctness).
- There are no tests for `clear_cache_muni()` or `clear_cache_tracts()`.
- No tests for `cnefe_dictionary()` or `cnefe_doc()`.
- No tests for the `crs_output` parameter behavior.
- The file `test-test-tracts_to_h3.R` has a double “test-” prefix, likely a typo.

Response: All five points accepted; two had already been addressed earlier in this revision.

Output correctness for `tracts_to_polygon()`. The file tested only input validation. It now has a suite mirroring the mocked setup of the `tracts_to_h3()` test, designed so the expected values are exact: two tracts, only one containing CNEFE points, so a covering polygon must receive exactly 100 of the 150 available and the other 50 must stay out. It also covers a tract split 50/50 across two polygons, preservation of the caller's columns and CRS, absence of the internal join column, and the two-stage diagnostics. Coverage of that file goes from 10.8% to 83.3%.

`clear_cache_muni()` and `clear_cache_tracts()`. Correct at the time of the report. They gained tests earlier in this revision, when the cache was segregated by census edition, covering both the per-edition and the all-editions paths.

`cnefe_dictionary()` and `cnefe_doc()`. They were the only exported functions with no tests at all, at 0%, because both end in `utils::browseURL()`. That call is now mocked, so the tests assert the resolved path, the invisible return and the rejection of an unsupported edition without opening a spreadsheet or a PDF viewer. The file goes from 0% to 76.9%.

`crs_output` behaviour. Only the rejection of an invalid CRS was covered, not what the argument does. There are now tests asserting that the result is reprojected, that the values are unchanged by reprojection, and that a projected input comes back in its own CRS and not in 4326.

The double test- prefix. It was a typo. Renamed.

We also made the test mocks more robust. Three mocks in that test named every formal of the internal function they replaced, so each new internal argument broke them. They broke three times during this revision. They now name only what they read and absorb the rest, following the pattern the helper file already used.

Additionally, since the cache changed to gzipped CSV under R1.11, every fixture-based test was serving a `.zip`, so the format all users now get had no coverage with real records, while the legacy path had all of it. The fixture now ships in both formats and defaults to the current one.

R2.C9: Portuguese comments

Portuguese comments. `utils-internal.R` lines 516 and 531 contain Portuguese comments (e.g., `# Ignorado, mantido para compatibilidade`, `# Usa piggyback para download dos assets do SC`); `tracts_to_h3.R`:503 has another (`# capture.output para capturar TUDO (output E messages)`). For consistency in an international codebase, these should be in English.

Response: Accepted. All three comments are now in English.

The comments in `utils-internal.R`:516 and :531 and in `tracts_to_h3.R` have been translated. They also went against our own contributing guidelines, which require English comments.

R2.C10: `tracts_to_h3()` uses unnecessary `exists()` check

`tracts_to_h3()` uses unnecessary `exists()` check. Lines 65–68 check if `(exists(".normalize_code_muni", mode = "function"))` before calling it, with a fallback to `as.integer()`. This function is always available in the package namespace, making the check unnecessary — likely a leftover from development.

Response: Accepted. The `exists()` check has been removed.

It was a leftover from development. `.normalize_code_muni()` is always reachable in the package namespace, so the `exists()` guard could never take its `as.integer()` branch, leaving an untestable fallback next to the real validation. Both the guard and the fallback are gone and the function is called directly.

References

- Graham, H. (2026). *a5R: ‘A5’ Discrete Global Grid System*. R package version 0.6.0. <https://CRAN.R-project.org/package=a5R>
- Instituto Brasileiro de Geografia e Estatística (IBGE) (2024). *Censo Demográfico 2022: Coordenadas Geográficas dos Endereços. Nota metodológica n. 01*. Rio de Janeiro: IBGE. <https://biblioteca.ibge.gov.br/visualizacao/livros/liv102063.pdf>
- Kmoch, A., Sahr, K., Chan, W. T., and Uemaa, E. (2025). IGEO7: A new hierarchically indexed hexagonal equal-area discrete global grid system. *AGILE: GIScience Series*, 6, 1–6. <https://doi.org/10.5194/agile-giss-6-32-2025>
- Massey, D. S. (2001). The prodigal paradigm returns: ecology comes back to sociology. In A. Booth and A. C. Crouter (Eds.), *Does It Take a Village? Community Effects on Children, Adolescents, and Families*. Mahwah, NJ: Lawrence Erlbaum.
- Pedreira Junior, J. U., Louro, T. V., de Assis, L. B. M., and Brito, P. L. (2025). Measuring land use mix with address-level census data. *engrXiv preprint*. <https://doi.org/10.31224/5975>
- Pedreira Junior, J. U., Louro, T. V., de Assis, L. B. M., Brito, P. L., and Bomfim, F. G. (2026). BGBI: A citywide-referenced and bidirectional land use mix index for planning and policy evaluation. *Land Use Policy*, 169, 108135. <https://doi.org/10.1016/j.landusepol.2026.108135>
- Pereira, R. H. M., and Barbosa, R. J. (2026). censobr: Fácil acesso aos dados dos censos demográficos brasileiros. *Dados*, 69(1), e20240150. <https://doi.org/10.1590/dados.2026.69.1.006>
- Song, Y., Merlin, L., and Rodriguez, D. (2013). Comparing measures of urban land use mix. *Computers, Environment and Urban Systems*, 42, 1–13. <https://doi.org/10.1016/j.compenvurbsys.2013.08.001>